Name : Manaswi Kabadi
Class : D20C
Roll no. : 15

Blockchain Experiment - 7

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory

Ganache is a personal Ethereum blockchain used for testing and development. It lets developers deploy contracts, build decentralized applications (DApps), and run tests in a safe, deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- A graphical interface to monitor blockchain activity

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Metamask is a browser wallet that signs transactions and connects the browser to an Ethereum network, including a custom local RPC such as Ganache.

Remix IDE is a browser-based environment for writing, compiling, deploying and testing Solidity contracts. With the *Injected Provider* environment, it sends transactions through Metamask.

Smart contract used: CertRegistry is an on-chain registry for academic certificates. The certificate file itself is encrypted off-chain and stored on IPFS. Only its IPFS CID and a keccak256 hash of the ciphertext are stored on the blockchain. This keeps the data private while making any tampering detectable.

Steps to Connect Ganache with Metamask and Remix IDE

1. **Install Ganache.** Download it from the Truffle Suite website and launch it to start a local blockchain with test Ether accounts.
2. **Connect Ganache with Metamask.**
    - Open Metamask and add a custom network.
    - Enter the Ganache RPC URL (e.g., http://127.0.0.1:7545) and the chain ID shown in Ganache.
    - Import a Ganache account into Metamask using its private key.
3. **Connect Remix IDE with Metamask.**
    - Open Remix IDE in the browser.
    - In the "Deploy & Run Transactions" tab, select *Injected Provider – Metamask* as the environment.
    - Remix connects to the Metamask account linked to Ganache.
4. **Create the smart contract.** Create CertRegistry.sol in Remix and paste the contract code.

5. **Compile and deploy.**
    - Compile with Solidity compiler 0.8.24 or higher.
    - Deploy CertRegistry (the constructor takes no arguments) and approve the transaction in Metamask.
6. **Check transaction details on Ganache.** View the mined blocks, the contract creation transaction and the balances in the Ganache interface.
7. **Interact with the contract in Remix.**
    - Call issueCertificate(certId, cid, dataHash) to anchor a certificate.
    - Call verifyCertificate(certId, dataHash) to check its status.
    - Call revokeCertificate(certId, reason) to revoke it.
    - Observe the state changes and the corresponding transactions in Ganache.

Outcome

- Understood how to link Metamask with Remix IDE for deploying Ethereum smart contracts.
- Successfully deployed and interacted with the CertRegistry smart contract through Ganache.
- Verified all transactions and balances within the Ganache environment.

Implementation

CertRegistry.sol

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.24;

contract CertRegistry {

    /*//////////////////////////////////////////////////////////////
                                TYPES
    //////////////////////////////////////////////////////////////*/

    /// @notice Result of a verification attempt.
    enum Status {
        NotFound, // no certificate with this id was ever issued
        Valid, // issued, not revoked, and the supplied digest matches
        Tampered, // issued, but the supplied digest does not match
        Revoked // issued and digest matches, but later revoked
    }

    /**
     * @param cid IPFS content identifier of the encrypted certificate.
     * @param dataHash keccak256 of the raw ciphertext bytes stored at `cid`.
     * @param issuer Address that called issueCertificate.
     * @param issuedAt Block timestamp of issuance.
     * @param revokedAt Block timestamp of revocation, 0 while still valid.
     */
    struct Cert {
        string cid;
        bytes32 dataHash;
        address issuer;
        uint64 issuedAt;
```

```
    uint64 revokedAt;
  }
  /*//////////////////////////////////////////////////////////////
                        STORAGE
  //////////////////////////////////////////////////////////////*/
  /// @notice Contract owner; the only account that can manage issuers.
  address public owner;
  /// @notice Whitelisted issuing authorities (universities, exam boards).
  mapping(address => bool) public isIssuer;
  /// @notice certId => certificate record.
  mapping(bytes32 => Cert) private _certs;

    /// @notice Number of certificates issued so far (the paper's counter).
    uint256 public totalIssued;
    /*//////////////////////////////////////////////////////////////
                        EVENTS
    //////////////////////////////////////////////////////////////*/
    event IssuerAdded(address indexed account);
    event IssuerRemoved(address indexed account);
    event OwnershipTransferred(address indexed from, address indexed to);

   event CertificateIssued(
        bytes32 indexed certId,
        address indexed issuer,
        string cid,
        bytes32 dataHash,
        uint64 issuedAt
    );
    event CertificateRevoked(
        bytes32 indexed certId,
        address indexed issuer,
        string reason,
        uint64 revokedAt
    );
/*//////////////////////////////////////////////////////////////
                        ERRORS
//////////////////////////////////////////////////////////////*/
error NotOwner();
error NotIssuer();
error ZeroAddress();
error EmptyCid();
error ZeroHash();
error ZeroCertId();
 error AlreadyIssued(bytes32 certId); error UnknownCertificate(bytes32 certId); error AlreadyRevoked(bytes32 certId);
/*//////////////////////////////////////////////////////////////
                        MODIFIERS
//////////////////////////////////////////////////////////////*/
```

```
    modifier onlyOwner() {
        if (msg.sender != owner) revert NotOwner();
        _;
    }

    modifier onlyIssuer() {
        if (!isIssuer[msg.sender]) revert NotIssuer();
        _;
    }

    /*//////////////////////////////////////////////////////////////
                            CONSTRUCTION
    //////////////////////////////////////////////////////////////*/

    /// @notice Deployer becomes owner and the first issuer.
    constructor() {
        owner = msg.sender;
        isIssuer[msg.sender] = true;
        emit OwnershipTransferred(address(0), msg.sender);
        emit IssuerAdded(msg.sender);
    }

    /*//////////////////////////////////////////////////////////////
                            ADMIN FUNCTIONS
    //////////////////////////////////////////////////////////////*/

    function addIssuer(address account) external onlyOwner {
        if (account == address(0)) revert ZeroAddress();
        if (isIssuer[account]) return; // idempotent, no event spam
        isIssuer[account] = true;
        emit IssuerAdded(account);
    }

    function removeIssuer(address account) external onlyOwner {
        if (!isIssuer[account]) return;
        isIssuer[account] = false;
        emit IssuerRemoved(account);
    }

    function transferOwnership(address newOwner) external onlyOwner {
        if (newOwner == address(0)) revert ZeroAddress();
        address old = owner;
        owner = newOwner;
        emit OwnershipTransferred(old, newOwner);
    }

    /*//////////////////////////////////////////////////////////////
                            CERTIFICATE LIFECYCLE
    //////////////////////////////////////////////////////////////*/

    /**
     * @notice Anchor a new certificate. This is the paper's `issue()`.
     * @param certId Caller-computed id, normally
     * keccak256(abi.encode(studentId, course, issueDate)).
     * Computing it off-chain keeps the student's identifiers
```

```
    * off the public chain while staying
deterministic.

    * @param cid IPFS CID of the encrypted
certificate.
             * @param dataHash keccak256 of the
        ciphertext bytes stored at `cid`.

         */

        function issueCertificate(

            bytes32 certId,

            string calldata cid,

            bytes32 dataHash

        ) external onlyIssuer {

                if (certId == bytes32(0)) revert
        ZeroCertId();

            if (bytes(cid).length == 0) revert
        EmptyCid();

            if (dataHash == bytes32(0)) revert
        ZeroHash();

            if (_certs[certId].issuedAt != 0) revert
        AlreadyIssued(certId);

            uint64 ts = uint64(block.timestamp);

            _certs[certId] = Cert({

                cid: cid,

                  dataHash: dataHash,

                issuer: msg.sender,

                issuedAt: ts,

                revokedAt: 0

            });


            unchecked {
                    ++totalIssued;
            }

            emit CertificateIssued(certId, msg.sender,
cid, dataHash, ts);

        }

        /**

         * @notice Revoke a certificate. This is the
paper's `revoke()`.

         * @dev Only the original issuer may revoke,
so one university cannot

         * invalidate another's degrees. The record is
kept, never deleted:

         * an employer must still be able to see that
the certificate existed

         * and was withdrawn.

         */

        function revokeCertificate(

            bytes32 certId,

            string calldata reason

        ) external onlyIssuer {

            Cert storage c = _certs[certId];

            if (c.issuedAt == 0) revert
UnknownCertificate(certId);

            if (c.issuer != msg.sender) revert
NotIssuer();

            if (c.revokedAt != 0) revert
AlreadyRevoked(certId);

            uint64 ts = uint64(block.timestamp);

            c.revokedAt = ts;

                emit CertificateRevoked(certId,
            msg.sender, reason, ts);
```

```
    }

    /*//////////////////////////////////////////////////////////////
                            VIEW / VERIFICATION
    //////////////////////////////////////////////////////////////*/

    /**
     * @notice Verify a certificate. This is the paper's `verify()`.
     * @dev Free to call: the verifier downloads the ciphertext from IPFS,
     * hashes it locally, and passes the digest here. A mismatch means
     * the off-chain file was altered after issuance.
     * @return status Outcome of the check.
     * @return cert The stored record (zeroed when status is NotFound).
     */
    function verifyCertificate(
        bytes32 certId,
        bytes32 dataHash
    ) external view returns (Status status, Cert memory cert) {
        cert = _certs[certId];
        if (cert.issuedAt == 0) return (Status.NotFound, cert);
        if (cert.dataHash != dataHash) return (Status.Tampered, cert);
        if (cert.revokedAt != 0) return (Status.Revoked, cert);
        return (Status.Valid, cert);
    }

    /// @notice Read a record without supplying a digest. The paper's `view()`.
    function getCertificate(
        bytes32 certId
    ) external view returns (Cert memory) {
        Cert memory c = _certs[certId];
        if (c.issuedAt == 0) revert UnknownCertificate(certId);
        return c;
    }

    /// @notice True if the certificate exists and has not been revoked.
    function isValid(bytes32 certId) external view returns (bool) {
        Cert storage c = _certs[certId];
        return c.issuedAt != 0 && c.revokedAt == 0;
    }

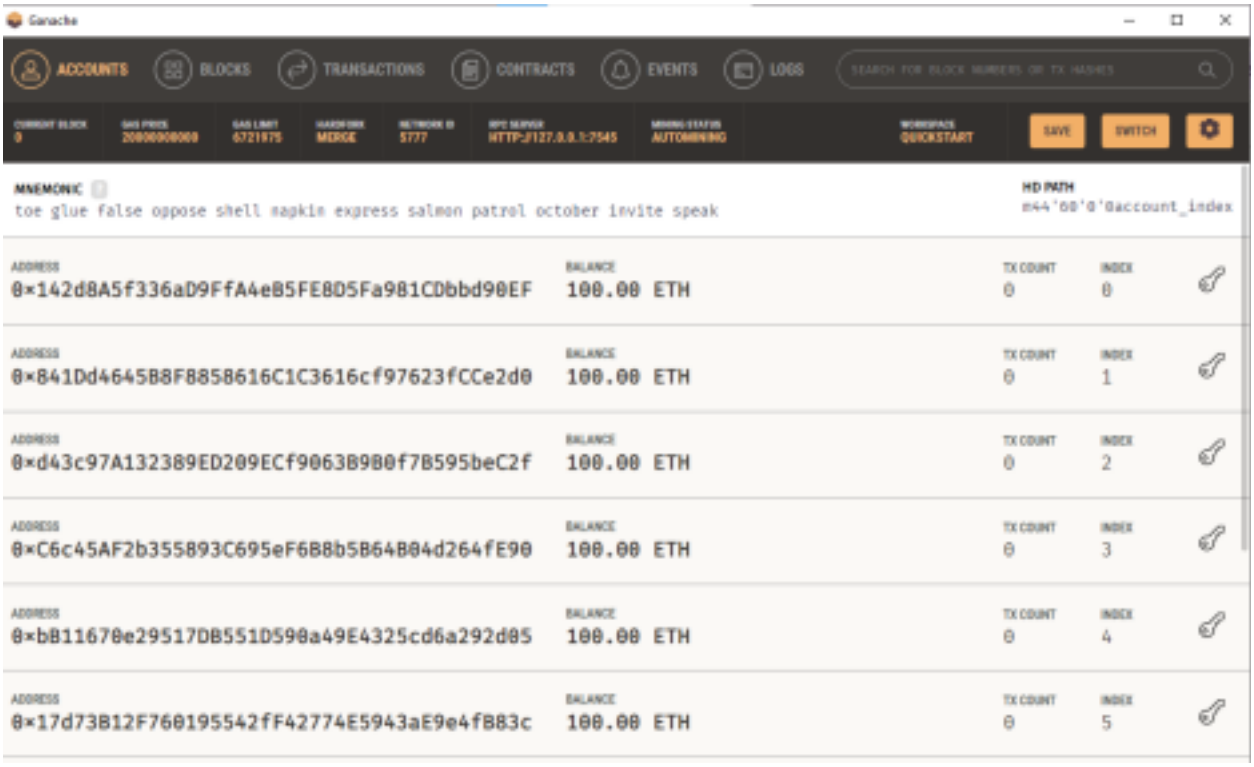
    /**
     * @notice Helper so the dApp and the tests derive certIds identically.
     * @dev Pure, so it costs nothing and can be called off-chain.
     */
    function computeCertId(
        string calldata studentId,
        string calldata course,
        string calldata issueDate
    ) external pure returns (bytes32) {
```

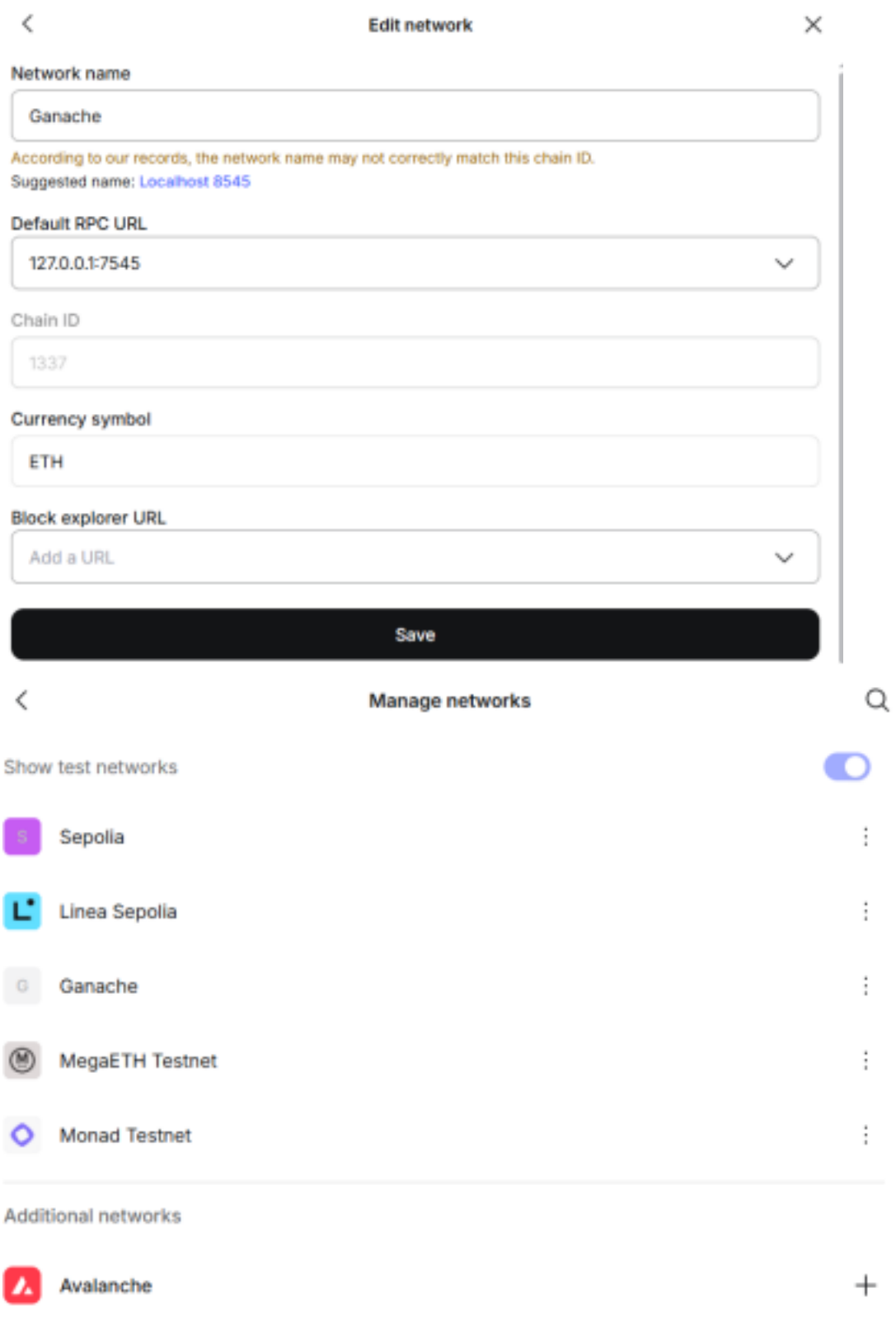
Description of the contract:

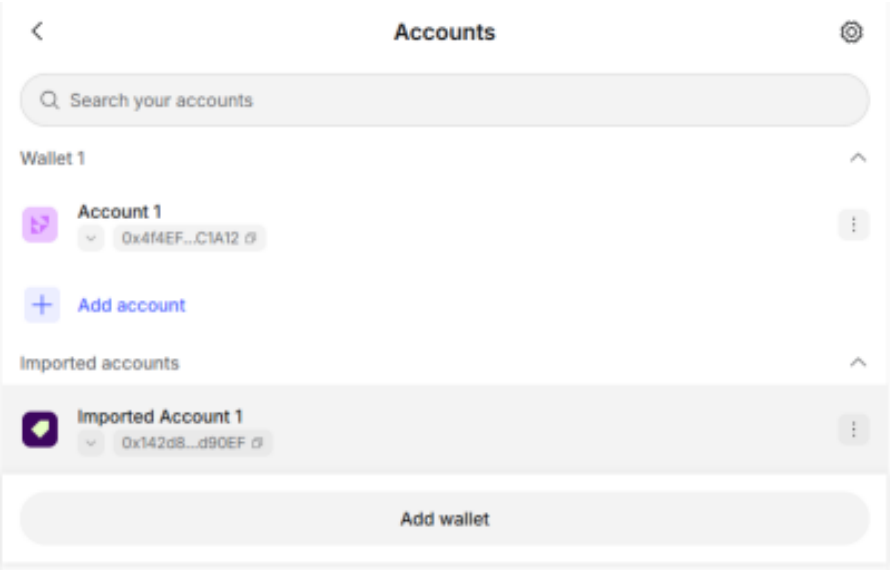
```
        return keccak256(abi.encode(studentId,                }
course, issueDate));
```

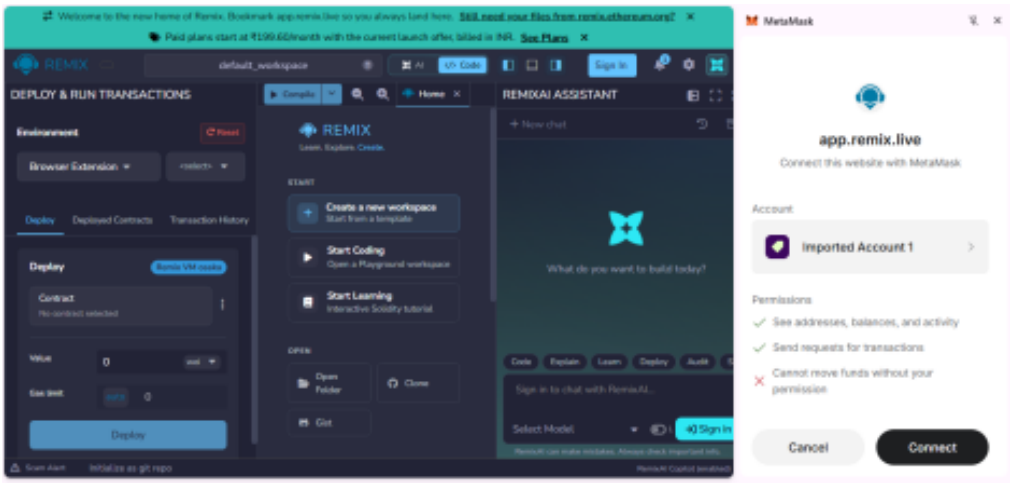
- **Roles:** The deployer becomes the owner and the first issuer. The owner can add or remove issuers using addIssuer() and removeIssuer().
- **issueCertificate():** Only an authorized issuer can call it. It stores the CID, data hash, issuer address and timestamp against a unique certId, and emits the CertificateIssued event. Duplicate IDs are rejected.
- **revokeCertificate():** Only the original issuer can revoke. The record is kept and marked with a revocation timestamp, not deleted.
- **verifyCertificate():** A free view function that returns one of four statuses: *NotFound*, *Valid*, *Tampered* (hash mismatch) or *Revoked*.
- **Helpers:** getCertificate(), isValid() and computeCertId() let the dApp and tests read records and derive IDs consistently.

Output




Edit network
Network name
Ganache
According to our records, the network name may not correctly match this chain ID.
Suggested name: Localhost 8545
Default RPC URL
127.0.0.1:7545
Chain ID
1337
Currency symbol
ETH
Block explorer URL
Add a URL
Save
Manage networks
Show test networks
Sepolia
Linea Sepolia
Ganache
MegaETH Testnet
Monad Testnet
Additional networks
Avalanche





Inference

- The contract deployed successfully on the local Ganache blockchain, and each state-changing call (issue, revoke, add issuer) appeared as a mined transaction with gas used and a reduced account balance.
- Read-only calls such as verifyCertificate() cost no gas and created no transactions.
- Only the hash and CID are stored on-chain, so the certificate data stays private.
- Verification returned *Valid* for the original hash and *Tampered* for a modified one. After revocation it returned *Revoked*.
- Access control worked: non-issuer accounts could not issue or revoke certificates.

Conclusion

Ganache provides a simple and powerful environment to simulate a local Ethereum blockchain. By integrating it with Metamask and Remix, the CertRegistry contract was deployed, tested and verified without spending real Ether. The experiment showed how blockchain can reduce certificate fraud by providing tamper-proof, publicly verifiable records.